



Факультет экономических наук

Базовая кафедра инфраструктуры
финансовых рынков

Москва

DOM-дерево HTML

Структура веб-страницы и поиск элементов для последующего парсинга

Бакланова Валерия Сергеевна

преподаватель БКИФР, младший научный сотрудник ЦФИАНД



DOM (Document Object Model, объектная модель документа) — набор правил, по которым браузер строит структуру веб-страницы на основе HTML-разметки.

DOM хранит содержимое страницы в виде дерева: каждый HTML-тег становится узлом этого дерева, а вложенные теги — его потомками.

Важно: узлы могут быть вложены сколь угодно глубоко, но чем больше уровней вложенности — тем сложнее ориентироваться в коде и тем медленнее работает страница.

Зачем это нужно при парсинге:

- DOM-дерево — это карта страницы, по которой мы находим нужный элемент и извлекаем его данные.
- Менять или создавать узлы не требуется — важно понимать структуру дерева и уметь получать доступ к нужному узлу и информации в нём.



Блочные элементы (block) формируют основную структуру страницы: занимают всю ширину контейнера и располагаются друг под другом.

Особенность	Что это значит для парсинга
Контейнеры	Часто вкладывают в себя другие элементы — удобно извлекать группы данных
CSS-классы	Обычно имеют уникальные классы/id — легко выбрать селектором
Стабильность	Реже меняются на динамических страницах — надёжная опора для парсера

- `<div>` — универсальный контейнер
- `<p>` — абзац
- `<h1>...<h6>` — заголовки
- `<ul>`, `<ol>`, `<li>` — списки и их пункты
- `<table>`, `<tr>`, `<td>` — таблицы
- `<header>`, `<footer>`, `<nav>`, `<section>` — смысловые блоки

Строчные элементы (inline) размечают фрагменты текста внутри блока и располагаются в потоке строки.

- `<a>` — ссылка `<span>` — контейнер для текста `<strong>`, `<b>` — важный текст `<em>`, `<i>` — выделенный текст
- Внутри строчных элементов нельзя вставлять блочные — только текст или другие строчные элементы.



```
<p class="text" id="unique">Учим парсинг вместе</p>
```

открывающий тег атрибуты (имя="значение") контент закрывающий тег

- **Открывающий тег** — имя элемента в угловых скобках; сигнализирует о начале элемента.
- **Закрывающий тег** — то же имя с косой чертой перед ним; отмечает конец элемента. Забытый закрывающий тег — частая ошибка новичков, ведущая к непредсказуемому результату.
- **Атрибут** — пара `имя="значение"` внутри открывающего тега (например, `class` или `id`); у элемента их может быть несколько.
- **Контент** — то, что находится между тегами; именно его мы, как правило, и парсим.



Если забыть закрывающий тег, браузер сам «достроит» структуру — но не всегда так, как вы ожидали:

```
<!DOCTYPE html>
<html><head>
  <title>Пример</title>
</head><body>
  <p>Это первый абзац.</p>
  <p>Это второй абзац
  с <strong>важным текстом.</strong>
  <p>Это третий абзац.
  ← нет </p> !
</body></html>
```

Как это увидит пользователь:

Это первый абзац.

Это второй абзац с **важным текстом.**

Это третий абзац.

У одного тега может быть сразу несколько атрибутов:

```
<p class="text" id="unique">Учим парсинг вместе</p>
```

имя атрибута = "значение атрибута"

имя атрибута = "значение атрибута"



Элемент на странице можно найти по тегу, атрибуту или их сочетанию:

Селектор	Что ищет
<code>id="name_id"</code>	элемент по идентификатору
<code>class="name_class"</code>	элемент по имени класса
<code><div></code>	элемент по имени тега
<code>href="link"</code>	элемент по значению атрибута

```
<div class="main author">
  <p class="text">
    Лев Толстой
  </p>
</div>
```

.main.author — составной селектор:
ищет элемент сразу с двумя классами

Совет: составные селекторы (два класса подряд без пробела) помогают точно выбрать элемент, когда один класс встречается на странице слишком часто.



Когда сложно подобрать уникальный CSS-селектор, на помощь приходит XPath — способ находить элементы по их пути в DOM-дереве.

XPath — мощный и гибкий инструмент, но при этом потенциально рискованный.

Плюс

Позволяет составлять очень точные и сложные запросы к любому элементу дерева — максимальная гибкость поиска.

Риск

Если разработчик сайта изменит структуру страницы, путь до элемента изменится — и парсер перестанет находить нужные данные.



Правило 1. Запрос XPath всегда начинается с символа / или //.

/ аналогичен символу > в CSS (прямой потомок), а // — пробелу в CSS (потомок на любом уровне вложенности).

`element1/element2`

element2 — прямой потомок
element1

`element1//element2`

element2 — потомок element1 на
любом уровне

```
<article>
  <header>
    <h2>Подзаголовок в статье</h2>
  </header>
</article>
```

```
//article/header/h2[text()='Подзаголовок в статье']
```



Правило 2. Квадратные скобки [] фильтруют найденные элементы по условию — например, по значению атрибута.

@ указывает, что дальше идёт имя атрибута, а строка в кавычках — его нужное значение.

```
//a[@id='a_back']
```

```
<div class="headers">...</div>
<a href="https://parsinger.ru/..."
  id="a_back">...</a>
<div class="main">...</div>
<div class="bottom"></div>
```

Найдёт ссылку с атрибутом

id="a_back"

Фильтровать можно по любому атрибуту: id, class, title, href и другим — по одному или в сочетании.



Полное совпадение — когда нужен элемент с точным текстом:

```
//button[text()='Купить']
```

Такой запрос сработает, только если текст внутри тега совпадает полностью — это не всегда удобно для многоязычных или динамических сайтов.

Частичное совпадение — функция `contains()` ищет по фрагменту текста или атрибута:

```
//p[contains(text(), "Артикул")]  
//li[contains(@id, "material")]
```



Правило 3. Символ * выбирает элемент любого типа — удобно, когда точный тег заранее неизвестен.

```
//div/*[@class="price"]
```

```
<div class="price_box">  
  <p class="price">2310 руб</p>  
</div>
```

Найдёт любой элемент с классом
"price"

внутри div — независимо от того, это <p>, или
другой тег.



Задача	CSS / XPath	Пример
По id	CSS	#name_id
По классу	CSS	.name_class
По двум классам	CSS	.main.author
Потомок на любом уровне	XPath	//tag1//tag2
По атрибуту	XPath	//a[@id='a_back']
По точному тексту	XPath	//button[text()='Купить']
По части текста	XPath	contains(text(), "...")
Любой тег с условием	XPath	//div/*[@class="price"]

Главное: начинайте с самого простого и стабильного селектора (id, класс) и переходите к XPath, когда CSS-селектора недостаточно.



Теперь, когда мы умеем находить элементы в DOM-дереве по CSS-селекторам и XPath, можно переходить к инструментам, которые делают это в коде на Python:

Requests

Библиотека для отправки HTTP-запросов и получения HTML-страниц с сайта.

BeautifulSoup

Библиотека для разбора HTML и поиска элементов по тегам, атрибутам и CSS-селекторам.

Selenium

Инструмент для управления браузером — нужен, когда страница строится через JavaScript.

На следующем занятии: знакомимся с библиотекой Requests и учимся получать HTML-код страницы для парсинга.